

Predizione della Malattia Epatica tramite Modelli di Classificazione e Apprendimento

Documentazione Progetto

Gruppo di lavoro

- Nicolò de Bari, 806301, n.debari4@studenti.uniba.it

https://github.com/nikegyz/Progetto_ICON_25-26

AA 2025-2026

Introduzione

Il dominio di interesse di questo progetto è la predizione della malattia epatica a partire da un insieme di esami del sangue. Il dataset scelto è l'Indian Liver Patient Dataset (ILPD), disponibile su Kaggle e raccolto nell'India del Nord-Est, spesso utilizzato per applicare metodi di classificazione con modelli di apprendimento supervisionato e non. Il dataset contiene 583 istanze di pazienti e una variabile target binaria, sulla quale eseguire la predizione, oltre agli attributi clinici su cui addestrare i modelli. La variabile target, chiamata Dataset, viene ricodificata internamente: il valore 1 indica la presenza di malattia epatica (Liver Patient) e il valore 2 viene mappato a 0 per indicare l'assenza (Non Liver Patient). Gli attributi di input selezionati sono i seguenti:

- Age : età del paziente (anni)
- Total_Bilirubin : bilirubina totale (mg/dL)
- Direct_Bilirubin : bilirubina diretta (mg/dL)
- Alkaline_Phosphotase : fosfatasi alcalina (IU/L)
- Alamine_Aminotransferase : alanina aminotransferasi, ALT (IU/L)
- Aspartate_Aminotransferase : aspartato aminotransferasi, AST (IU/L)
- Total_Protiens : proteine totali (g/dL), il nome riprende il refuso presente nel CSV originale
- Albumin : albumina (g/dL)
- Albumin_and_Globulin_Ratio : rapporto albumina/globulina

La colonna Gender non è stata inclusa tra le feature, per concentrare i modelli sui parametri ematochimici. Il dataset è sbilanciato: 416 casi positivi (circa il 71%) contro 167 negativi (circa il 29%). Per questo motivo, come si vedrà nelle analisi, la balanced accuracy e l'AUC risultano più informative della sola accuracy. I valori mancanti vengono sostituiti con la media della rispettiva colonna, mentre la suddivisione train/test è impostata al 70% e 30% con un seed fisso (random_state=1) per garantire la riproducibilità

Sommario

Il Knowledge Base System realizzato è composto da due parti principali: la Knowledge Base (KB) e il modulo di classificazione delle classi del dataset. La KB prende in input un file di testo contenente le regole e le affermazioni iniziali sul dominio medico e inferisce nuova conoscenza, restituendo le informazioni richieste durante la fase di test. La KB possiede tre azioni principali: Assert, Ask e Retract. Assert serve ad aggiungere affermazioni e regole, Ask a porre query e ottenere affermazioni durante i test, e Retract a rimuovere affermazioni o regole. La classificazione viene eseguita con più modelli, supervisionati e non, elencati nel prossimo paragrafo. Una volta terminata, i valori delle metriche prodotte da ciascun modello vengono salvati come affermazioni in un file, che la KB legge e acquisisce come propria conoscenza. Successivamente vengono eseguiti dei test su queste affermazioni per verificare che la KB abbia assimilato la conoscenza in modo corretto. Il processo di classificazione viene avviato dalla KB attraverso l'azione Ask, inserendo il termine classification seguito dal nome della classe di interesse. Nel mio caso la query (classification patient) instrada la richiesta verso il dataset Indian Liver Patient e produce la classificazione binaria tra Liver Patient e Non Liver Patient. Per ogni componente con un comportamento casuale (rete neurale, Random

Forest, K-Means) ho impostato un seed fisso, in modo che i risultati restino stabili tra le diverse esecuzioni.

Elenco argomenti di interesse

- Case-Based Reasoning: KNN (Apprendimento Supervisionato)
- Decision Tree (Apprendimento Supervisionato)
- Random Forest (Apprendimento Supervisionato)
- Logistic Regression (Apprendimento Supervisionato)
- Neural Network (Apprendimento Supervisionato)
- Naive Bayes (Apprendimento Probabilistico)
- Hard Clustering: K-Means (Apprendimento Non Supervisionato)

K-Nearest Neighbors

Sommario

La classificazione si esegue separando gli attributi del dataset in due gruppi: gli attributi di input, di cui si conoscono sempre i valori, e l'attributo di output su cui effettuare la predizione. Nel mio caso gli attributi di input sono i nove parametri ematochimici (Age, Total_Bilirubin, Direct_Bilirubin, Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase, Total_Protiens, Albumin e Albumin_and_Globulin_Ratio), mentre l'attributo di output è la classe del paziente, ovvero Liver Patient nel caso di malattia epatica e Non Liver Patient in caso contrario. Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello K-Nearest Neighbors relative alla classe su cui si è operato.

Strumenti utilizzati

Il K-Nearest Neighbors è un caso di Case-Based Reasoning: dato un esempio da classificare, si cercano i k esempi memorizzati più simili e si assegna la classe alla quale appartiene la maggioranza di essi. Non vi è una vera fase di addestramento, perché il modello memorizza il training set e opera al momento della predizione. Per migliorare l'efficacia ho utilizzato la procedura K-Fold Cross Validation, che divide gli esempi in k gruppi (fold) di pari dimensione, usando k-1 fold per l'addestramento e il rimanente per la verifica.

Decisioni di Progetto

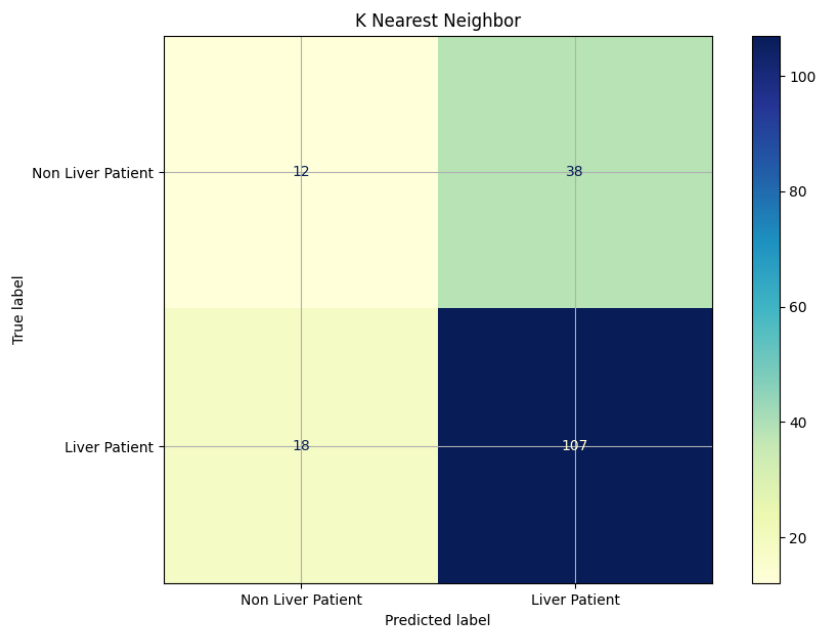
Per implementare il modello ho usato il modulo `KNeighborsClassifier` della libreria `sklearn.neighbors` e, per ottimizzarlo, la funzione `GridSearchCV` della libreria `sklearn.model_selection`. La griglia esplora il numero di vicini `n_neighbors` da 1 a 9 e il parametro `weights` tra i valori `uniform` e `distance`.

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione `Ask`. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga i e colonna j è il numero di casi in cui un esempio di classe i è stato classificato come classe j. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni.



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
K Nearest Neighbors	0.68	0.68	0.68	0.68	0.548	0.7355

Sul test set il KNN classifica correttamente 107 pazienti malati (veri positivi) e 12 pazienti sani (veri negativi), con 38 falsi positivi e 18 falsi negativi. L'accuracy si attesta al 68%, ma la balanced accuracy di 0.548 mostra che il modello fatica a riconoscere la classe minoritaria, cioè i pazienti sani. L'AUC di 0.736 indica comunque una discreta capacità di separazione a livello probabilistico.

DECISION TREE

Sommario

La classificazione si esegue separando gli attributi del dataset nei due gruppi già descritti: i nove attributi di input ematochimici e l'attributo di output, cioè la classe del paziente (Liver Patient oppure Non Liver Patient). Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello Decision Tree relative alla classe su cui si è operato.

Strumenti utilizzati

Un albero di decisione è costituito da nodi interni, che contengono condizioni sugli attributi, e da nodi foglia, che contengono la predizione della classe. La classificazione di un esempio parte dalla radice e percorre gli archi corrispondenti all'esito delle condizioni incontrate, fino a raggiungere una foglia. Anche in questo caso ho utilizzato la K-Fold Cross Validation per la scelta dei parametri.

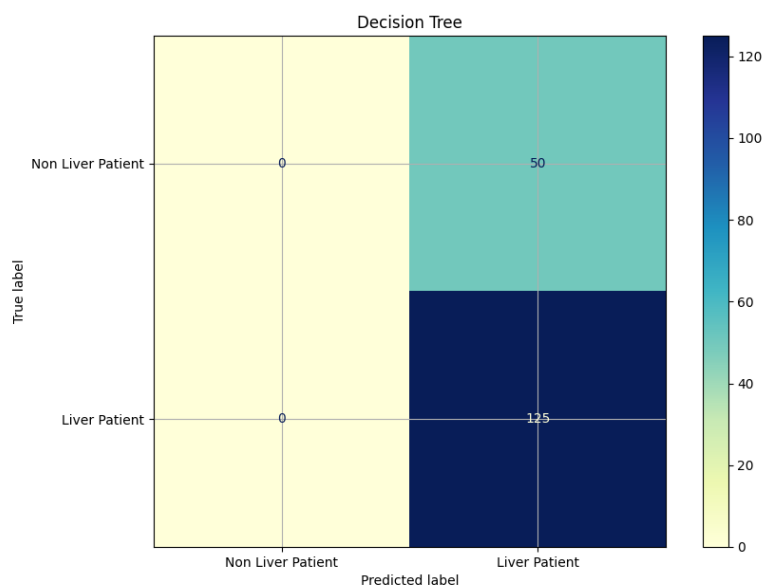
Decisioni di Progetto

Per implementare il modello ho usato il modulo `DecisionTreeClassifier` della libreria `sklearn.tree`, addestrandolo dapprima con `random_state=42` per ricavare la profondità e il numero di feature, e ottimizzandolo poi con `GridSearchCV`. La griglia varia `max_depth` con passo 2 e `max_features` sul numero di attributi disponibili.

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione Ask. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga *i* e colonna *j* è il numero di casi in cui un esempio di classe *i* è stato classificato come classe *j*. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
Decision Tree	0.7143	0.7143	0.7143	0.7143	0.50	0.686

L'albero di decisione ottiene un'accuracy del 71.4%, ma la matrice di confusione rivela un comportamento problematico: tutti gli esempi del test set vengono classificati come Liver Patient e i veri negativi sono pari a zero. La balanced accuracy scende quindi a 0.50, equivalente a una scelta casuale. Il motivo è lo sbilanciamento del dataset: assegnare sempre la classe maggioritaria massimizza l'accuracy, ma azzerla la capacità di riconoscere i pazienti sani. L'AUC di 0.686 segnala comunque una residua capacità di ordinamento sulle probabilità.

Random Forest

Sommario

La classificazione si esegue separando gli attributi del dataset nei due gruppi già descritti: i nove attributi di input ematochimici e l'attributo di output, cioè la classe del paziente (Liver Patient oppure Non Liver Patient). Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello Random Forest relative alla classe su cui si è operato.

Strumenti utilizzati

La Random Forest è un metodo di ensemble learning composto da più alberi di decisione: ogni albero viene addestrato su un sottocampione del training set e, per ogni esempio, la predizione finale si ottiene aggregando per voto di maggioranza le predizioni dei singoli alberi. Questa combinazione riduce la varianza rispetto al singolo albero. Anche qui ho usato la K-Fold Cross Validation per la scelta dei parametri.

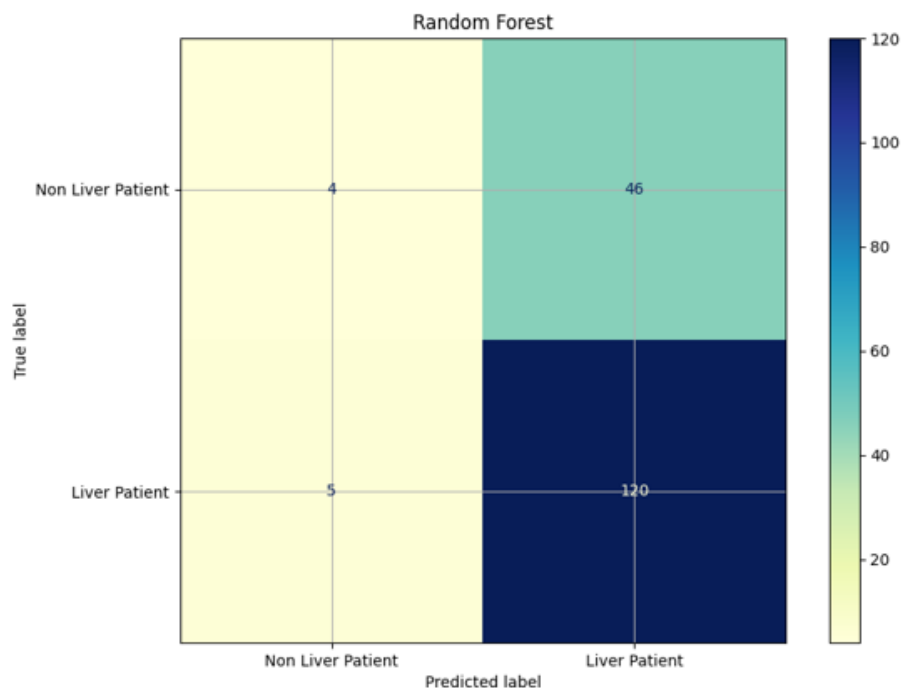
Decisioni di Progetto

Per implementare il modello ho usato il modulo `RandomForestClassifier` della libreria `sklearn.ensemble`, nella forma `RandomForestClassifier(random_state=42)`, lasciando la parallelizzazione (`n_jobs=-1`) alla `GridSearchCV`. La griglia varia `n_estimators` tra 15, 20, 30 e 40 e `max_depth` da 1 a 9.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione `Ask`. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga *i* e colonna *j* è il numero di casi in cui un esempio di classe *i* è stato classificato come classe *j*. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni.



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
Random Forest	0.7086	0.7086	0.7086	0.7086	0.52	0.7557

La Random Forest ottiene 120 veri positivi e 4 veri negativi, con 46 falsi positivi e 5 falsi negativi. La balanced accuracy di 0.52 è di poco superiore a quella dell'albero singolo, mentre l'AUC di 0.756 è la migliore tra i modelli ad albero: l'insieme di alberi conserva una buona capacità di ordinamento probabilistico anche in presenza di forte sbilanciamento

Logistic Regression

Sommario

La classificazione si esegue separando gli attributi del dataset nei due gruppi già descritti: i nove attributi di input ematochimici e l'attributo di output, cioè la classe del paziente (Liver Patient oppure Non Liver Patient). Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello Logistic Regression relative alla classe su cui si è operato.

Strumenti utilizzati

La Logistic Regression è un classificatore basato sulla regressione logistica, che stima la probabilità con cui un'osservazione appartiene a una delle classi della feature di output in base ai valori delle feature note. Poiché la classe è dicotomica (Liver Patient o Non Liver Patient), il modello è adatto a questo problema. Prima dell'addestramento i dati vengono normalizzati e, anche in questo caso, ho usato la K-Fold Cross Validation per la scelta dei parametri.

Decisioni di Progetto

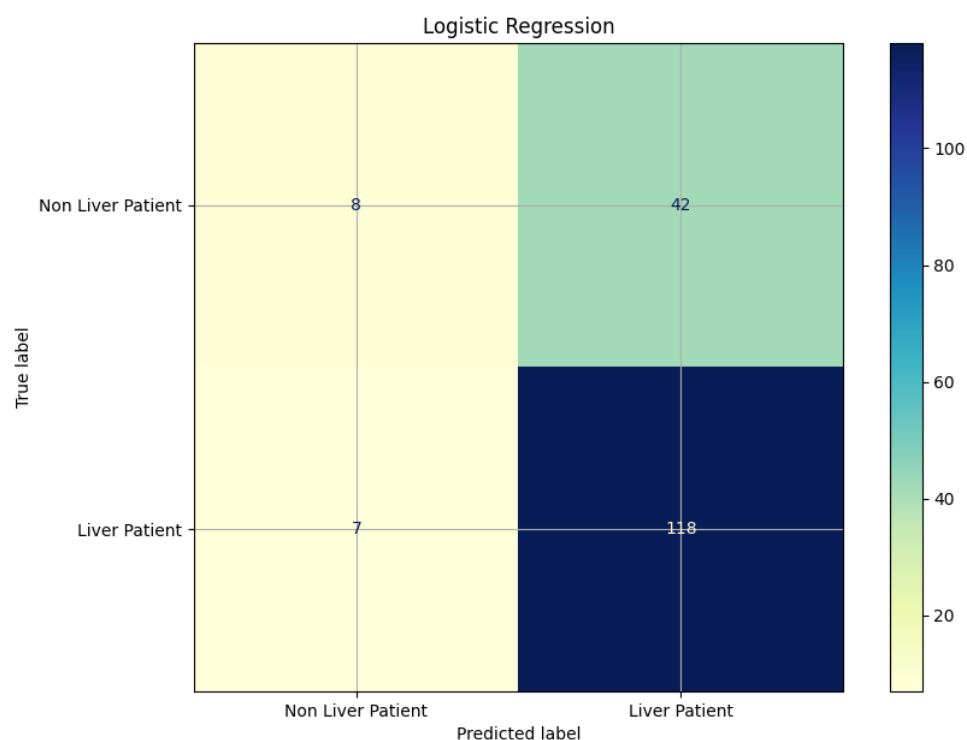
Per implementare il modello ho usato il modulo LogisticRegression della libreria `sklearn.linear_model` e, per normalizzare i valori, il modulo `StandardScaler` della libreria `sklearn.preprocessing`. L'ottimizzazione avviene con `GridSearchCV` a 5 fold, variando il parametro di regolarizzazione C su scala logaritmica e il solver tra `newton-cg`, `lbfgs` e `liblinear`, con penalità L2.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione Ask. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga i e colonna j è il numero di casi in cui un esempio di classe i è stato classificato come classe j. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni.



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
Logistic Regression	0.72	0.72	0.72	0.72	0.552	0.7501

La regressione logistica raggiunge l'accuracy più alta tra i modelli supervisionati (72%), con 118 veri positivi, 8 veri negativi, 42 falsi positivi e 7 falsi negativi. La balanced accuracy resta però bassa (0.552): anche questo modello tende a privilegiare la classe maggioritaria. L'AUC è pari a 0.750. È interessante notare come un modello lineare così semplice raggiunga prestazioni vicine a quelle della rete neurale, segno che la relazione tra feature e classe è in buona parte lineare.

Neural Network

Sommario

La classificazione si esegue separando gli attributi del dataset nei due gruppi già descritti: i nove attributi di input ematochimici e l'attributo di output, cioè la classe del paziente (Liver Patient oppure Non Liver Patient). Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello Neural Network relative alla classe su cui si è operato.

Strumenti utilizzati

La rete neurale è un modello computazionale non lineare composto da unità artificiali ispirate ai neuroni biologici, organizzate in unità di input, unità nascoste e unità di output. Le unità di input alimentano gli strati nascosti, che a loro volta determinano le predizioni dello strato di output. Nel mio caso si tratta di un perceptrone a più strati che ottimizza la funzione log-loss con l'ottimizzatore L-BFGS. Prima dell'addestramento i dati vengono normalizzati.

Decisioni di Progetto

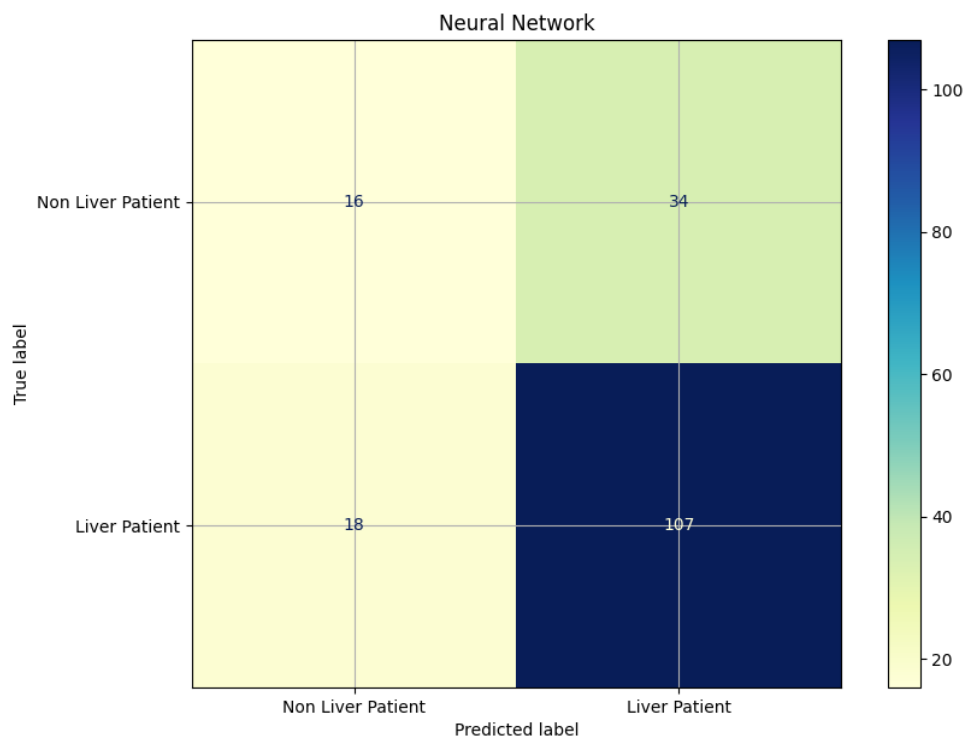
Per implementare il modello ho usato il modulo `MLPClassifier` della libreria `sklearn.neural_network`, con normalizzazione tramite `MinMaxScaler`. Trattandosi di un dataset di dimensioni contenute, ho scelto parametri adeguati: tre strati nascosti da 4 unità (`hidden_layer_sizes=(4,4,4)`), attivazione `relu`, solver `lbfgs` (consigliato per dataset piccoli) e `max_iter` tra 2000 e 2100. Ho inoltre impostato `random_state=42` per fissare l'inizializzazione casuale dei pesi. L'ottimizzazione è affidata a `GridSearchCV`.

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione Ask. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga *i* e colonna *j* è il numero di casi in cui un esempio di classe *i* è stato classificato come classe *j*. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni.



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
Neural Network	0.7029	0.7029	0.7029	0.7029	0.588	0.7449

La rete neurale classifica correttamente 107 pazienti malati e 16 pazienti sani, con 34 falsi positivi e 18 falsi negativi. L'accuracy è del 70.3% e la balanced accuracy di 0.588 è la seconda più alta dopo il Naive Bayes, indice di un equilibrio fra le due classi migliore rispetto ai modelli ad albero. L'AUC è pari a 0.745. Va segnalato che, a seconda dell'hardware, durante l'addestramento possono comparire avvisi sul numero di iterazioni e sulla convergenza, che però non incidono in modo evidente sui risultati.

Naive Bayes

Sommario

La classificazione si esegue separando gli attributi del dataset nei due gruppi già descritti: i nove attributi di input ematochimici e l'attributo di output, cioè la classe del paziente (Liver Patient oppure Non Liver Patient). Il dataset viene poi suddiviso, tramite la procedura `train_test_split`, in quattro insiemi: `X_train`, `X_test`, `y_train` e `y_test`. Una volta eseguita la classificazione viene creato il file `classification_statements`, che contiene le asserzioni derivate dai risultati; la Knowledge Base legge questo file e, tramite l'azione `Assert`, acquisisce le metriche prodotte dal modello Naive Bayes relative alla classe su cui si è operato.

Strumenti utilizzati

Il Naive Bayes è un classificatore probabilistico fondato sul teorema di Bayes, il cui obiettivo è imparare la dipendenza delle feature dalla classe per predire la classificazione di nuovi esempi. Il modello assume l'indipendenza condizionale tra le feature data la classe e stima le probabilità a priori di ciascuna classe e le probabilità condizionali di ogni feature, combinandole per produrre la predizione.

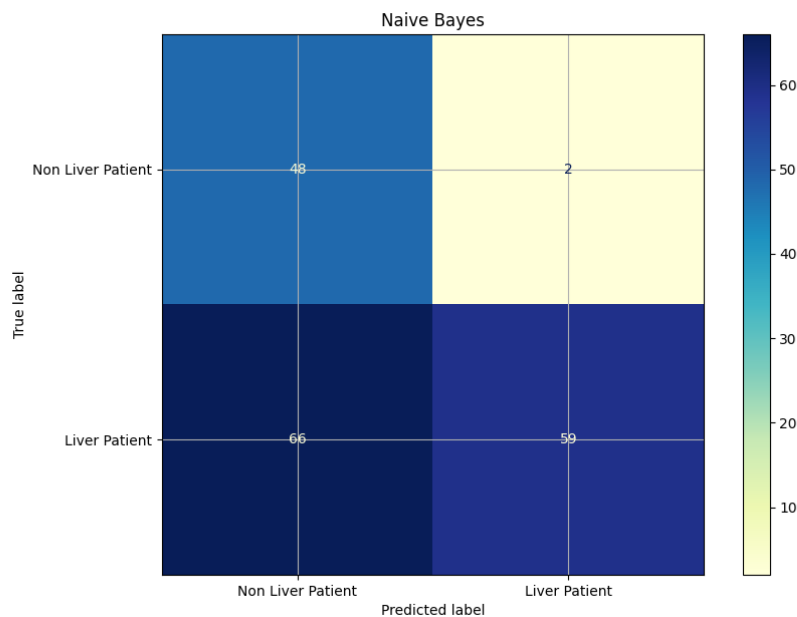
Decisioni di Progetto

Per implementare il modello ho usato il modulo `GaussianNB` della libreria `sklearn.naive_bayes`. Tra le diverse varianti di Naive Bayes disponibili, ho scelto la `GaussianNB` perché modella le feature come variabili continue con distribuzione gaussiana, scelta coerente con la natura continua dei valori ematochimici del dataset (bilirubina, albumina, enzimi epatici). Il modello viene addestrato direttamente, senza ricerca di iperparametri.

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

Valutazione

Le metriche usate per misurare l'efficacia della classificazione sono le stesse adottate per tutti i modelli: Accuracy, F1 Score, Precision, Recall, Balanced Accuracy e AUC Score. Tutte appartengono al modulo `sklearn.metrics` e, una volta inserite nella KB, possono essere interrogate con l'azione `Ask`. La matrice di confusione riportata di seguito rappresenta l'accuratezza della classificazione: le colonne indicano i valori predetti e le righe i valori reali, quindi l'elemento sulla riga `i` e colonna `j` è il numero di casi in cui un esempio di classe `i` è stato classificato come classe `j`. La classificazione avviene su un test set di circa 175 esempi, pari al 30% del dataset; grazie all'uso di un seed fisso i valori riportati sono riproducibili tra le esecuzioni.



	accuracy	f1_score	precision	recall	balanced_accuracy	auc
Naive Bayes	0.6114	0.6114	0.6114	0.6114	0.716	0.7754

Il Naive Bayes ottiene 48 veri negativi e 59 veri positivi, con soli 2 falsi positivi ma 66 falsi negativi. L'accuracy scende a 0.611, ma la balanced accuracy di 0.716 e l'AUC di 0.775 sono le più alte tra tutti i modelli: è l'unico classificatore che riconosce in modo affidabile i pazienti sani, ovvero la classe minoritaria. Il prezzo da pagare è un numero elevato di falsi negativi. In ambito clinico questo comportamento ha comunque valore, perché individuare correttamente la classe negativa è diagnosticamente utile.

K-Means

Sommario

Essendo un modello di apprendimento non supervisionato, la classificazione si esegue applicando prima uno scaler (MinMaxScaler) per normalizzare i valori, poi creando il modello con il numero di cluster desiderato e infine addestrandolo sull'intero dataset normalizzato. Una volta eseguita la classificazione viene creato il file `classification_statements`, che la KB legge per acquisire, tramite `Assert`, lo score relativo al clustering.

Strumenti utilizzati

Il K-Means è un algoritmo di hard clustering che riceve in input gli esempi e il numero di cluster e fornisce in output l'assegnazione di ogni esempio a un cluster. L'algoritmo converge verso un minimo locale riducendo iterativamente l'errore quadratico, poiché esiste solo un numero finito di possibili riassegnazioni.

Decisioni di Progetto

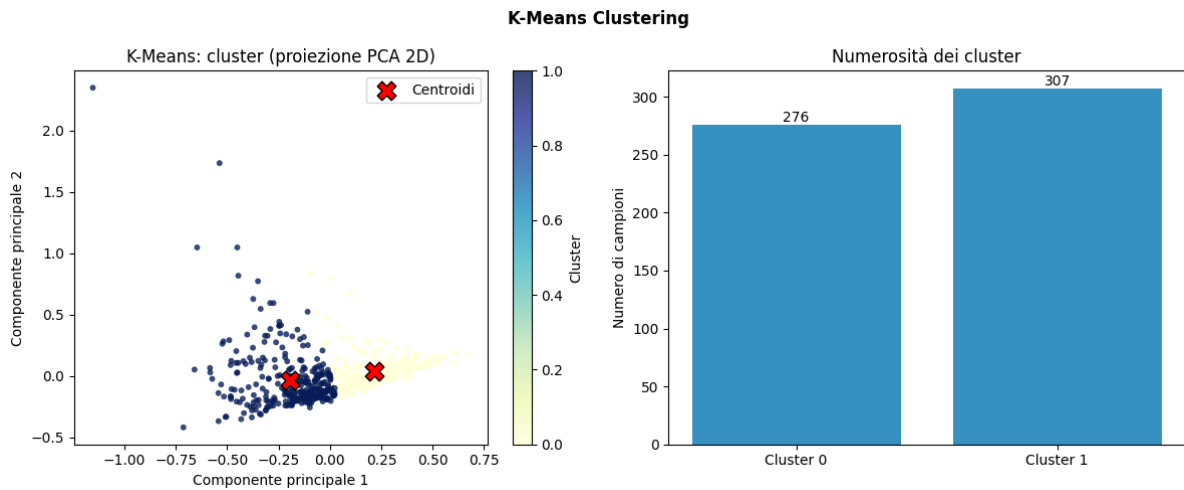
Per implementare l'algoritmo ho usato il modulo `KMeans` della libreria `sklearn.cluster`, con normalizzazione tramite `MinMaxScaler`. Ho impostato due cluster (`n_clusters=2`), coerenti con la classificazione binaria, `n_init=10` e `random_state=42` per rendere stabile l'inizializzazione dei centroidi. Trattandosi di un modello non supervisionato, non si applica la ricerca degli iperparametri con cross validation.

<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

Valutazione

La metrica usata per il K-Means è lo score, cioè il numero di esempi assegnati a ciascun cluster. Il clustering avviene su tutto il dataset (583 esempi). Per visualizzare il risultato, che riguarda nove dimensioni, ho proiettato i dati in due dimensioni con una PCA, colorando i punti per cluster e marcando i centroidi; accanto è riportata la numerosità dei due cluster.



Il K-Means suddivide gli esempi in due gruppi: 276 nel cluster 0 e 307 nel cluster 1. Poiché il dataset contiene il 71% di pazienti malati, questa ripartizione quasi paritaria non corrisponde alla distribuzione reale delle classi. Il K-Means infatti separa lo spazio geometrico delle feature e non i confini clinici. Per questo non gli si associa un'accuracy: lo score salvato nella KB è semplicemente il numero di esempi per cluster, verificabile nelle query dei test sul clustering.

Conclusioni

Per confrontare i risultati riporto di seguito la tabella con le metriche di tutti i modelli supervisionati e, separatamente, lo score del K-Means.

Modello	accuracy	f1_score	precision	recall	balanced_accuracy	auc
K Nearest Neighbors	0.680	0.680	0.680	0.680	0.548	0.736
Decision Tree	0.714	0.714	0.714	0.714	0.500	0.686
Random Forest	0.709	0.709	0.709	0.709	0.520	0.756
Naive Bayes	0.611	0.611	0.611	0.611	0.716	0.775
Neural Network	0.703	0.703	0.703	0.703	0.588	0.745
Logistic Regression	0.720	0.720	0.720	0.720	0.552	0.750

Score del K-Means: cluster 0 con 276 esempi e cluster 1 con 307 esempi. 0.552 0.750 Un primo dato da osservare è che accuracy, F1, precision e recall coincidono per ogni modello. Questo dipende dall'uso del parametro average=macro su un problema binario, dove queste quattro metriche collasano sull'accuracy globale. Gli indicatori realmente discriminanti sono quindi la balanced accuracy e l'AUC. Letti sotto questa luce, i risultati cambiano prospettiva rispetto alla sola accuracy. Il Naive Bayes, che sull'accuracy appare il modello peggiore (0.611), è in realtà il migliore nel distinguere effettivamente le due classi, con la balanced accuracy più alta (0.716) e il miglior AUC (0.775). La Logistic Regression ottiene l'accuracy assoluta più alta (0.720) ma penalizza la classe minoritaria, mentre il Decision Tree mostra il caso limite, con balanced accuracy pari a 0.50 perché classifica tutti gli esempi come malati. La rete neurale e la Random Forest si collocano in posizione intermedia. La conclusione che traggio da questo confronto è che, su un dataset clinico sbilanciato come l'Indian Liver Patient, non esiste un modello migliore in assoluto: la scelta dipende dalla metrica e dall'obiettivo. Se conta riconoscere correttamente i pazienti sani, il Naive Bayes è la scelta più solida; se conta solo la percentuale di predizioni corrette, la Logistic Regression risulta preferibile, pur con i limiti evidenziati dalla balanced accuracy. Il sistema di forward chaining completa il progetto: consente di inferire automaticamente nuova conoscenza dai fatti del dominio e di verificare, tramite la suite di test e le operazioni di Retract, che le metriche inserite nella KB restino coerenti.

Riferimenti Bibliografici

[1] David L. Poole, Alan K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 2nd Edition, Cambridge University Press, 2017.

[2] https://scikit-learn.org/stable/modules/model_evaluation.html

[3] <https://www.kaggle.com/datasets/uciml/indian-liver-patient-records>